

LiteCUA: Computer as MCP Server for Computer-Use Agent on AIOS

Kai Mei Xi Zhu Hang Gao Shuhang Lin Yongfeng Zhang *

Abstract

We present AIOS 1.0, a novel platform designed to advance computer-use agent (CUA) capabilities through environmental contextualization. While existing approaches primarily focus on building more powerful agent frameworks or enhancing agent models, we identify a fundamental limitation: the semantic disconnect between how language models understand the world and how computer interfaces are structured. AIOS 1.0 addresses this challenge by transforming computers into contextual environments that language models can natively comprehend, implementing a Model Context Protocol (MCP) server architecture to abstract computer states and actions. This approach effectively decouples interface complexity from decision complexity, enabling agents to reason more effectively about computing environments. To demonstrate our platform’s effectiveness, we introduce LiteCUA, a lightweight computer-use agent built on AIOS 1.0 that achieves a 14.66% success rate on the OSWorld benchmark, outperforming several specialized agent frameworks despite its simple architecture. Our results suggest that contextualizing computer environments for language models represents a promising direction for developing more capable computer-use agents and advancing toward AGI that can interact with digital systems.

1 Introduction

Large language models (LLMs) [1, 31, 19, 9, 42] have achieved impressive leaps in understanding knowledge and reasoning [34, 46, 4, 16, 15, 6, 14], fuelling a new generation of LLM-powered agents that plan, observe, and act to interact with digital environments [30, 36, 10, 47]. Research demonstrates that, when coupled with suitable prompting or fine-tuning, LLM-based agents can effectively plan to fulfill complex tool calling [24, 26, 41] and programming [33, 43, 50] tasks.

Moving beyond calling external tools or executing generated code, researchers have begun exploring whether LLM agents can perform more complex computer-use tasks, leveraging their emerging planning and action capabilities. This exploration has catalyzed the development of benchmarks assessing LLM-based agents in computer-use scenarios across operating systems [39, 5], mobile devices [7, 29], and browsers [45, 52, 8, 17, 23]. Parallel efforts have focused on leveraging LLMs and vision-language models (VLMs) to perceive computer/mobile environments and execute actions such as clicking, typing, and dragging interactive elements to complete tasks. Current approaches to computer-use agents (CUAs) primarily follow two trajectories: (1) developing agent frameworks with specialized modules for critical subtasks like grounding [2, 11], search [48], and reflection [51, 32]; and (2) embedding agentic capabilities directly into LLMs/VLMs to create native agent models [27, 12]. Despite these advances, CUAs continue to exhibit significant limitations in their ability to navigate complex computing environments effectively.

These limitations stem from a fundamental challenge illustrated in Figure 1: a profound disconnect between how humans and AI agents perceive and interact with computing environments. In the current paradigm, researches on CUA predominantly focus on enhancing agent capabilities while largely overlooking the environments in which these agents operate. As the right part of Figure 1 illustrates,

*Corresponding author: Yongfeng Zhang, yongfeng.zhang@rutgers.edu, Department of Computer Science, Rutgers University, New Brunswick, NJ 08854

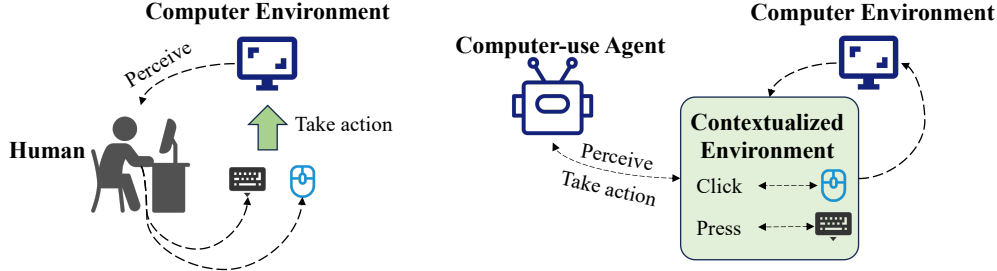


Figure 1: Illustration of the differences between how human operates computer and how computer-use agent (CUA) operates computer, where CUA requires a specific contextualized environment to understand the original computer environment and take actions.

there exists a significant opportunity to bridge this gap by creating agent-specific environments that mediate between the raw computer interface and the agent’s cognition. Rather than forcing agents to adapt to interfaces designed for human perception and action, we can transform these environments into contextualized representations that align with the semantic understanding of language models.

Inspired by this and based on the foundation laid out by the AIOS architecture [22, 28], we introduce **AIOS 1.0**, an augmented platform developed to move towards contextualization of computer environments for LLM-based agents. Specifically, AIOS 1.0 aims to model the full computer as MCP² server to provide LLM-friendly and well-structured context where interactive elements permissible actions are explicitly encoded in JSON schemas. This contextualization process transforms the computer from an opaque operational environment into a semantic landscape that aligns with the LLM’s understanding of the environment. This design equips agents with an informative yet compact view of the state space, enabling finer-grained reasoning over available operations and facilitating longer-horizon planning. By decoupling interface complexity from decision complexity, AIOS 1.0 addresses a fundamental bottleneck in current CUA capabilities. The contextualized environment acts as a semantic bridge between the agent’s language-based reasoning and the computer’s operational requirements.

Upon AIOS 1.0, we introduce **LiteCUA**, a computer-use agent that is developed by us with simple architectural design. LiteCUA achieves competitive results with several agent frameworks, which scores 14.66% in the OSWorld benchmark [39]. The success of LiteCUA emphasizes that AIOS 1.0’s approach of creating agent-specific environments can effectively reduce the implementation complexity required to build computer-use agents, allowing researchers to focus on advancing agent capabilities rather than wrestling with the intricacies of computer interface interpretation. By reframing the computer-use agent challenge as an environmental contextualization problem rather than solely a model capability issue, AIOS 1.0 aims to catalyze the next leap in computer-use agent capabilities. This approach represents a crucial step toward developing truly general computer-use agents, systems that can navigate the digital world with the flexibility and comprehension previously exclusive to human users.

2 Architecture and System Design

Following the architecture of AIOS [22, 28], which abstracts a specific AIOS kernel for isolating agent logic [28] and managing agent-related resources and services, we extend this architecture to create AIOS 1.0, a platform optimized for developing and running computer-use agents.

2.1 Architecture

Application Layer: In the Application Layer, agents are provided with SDK APIs to interact with some crucial components like Terminal, Code editor, Browser, and Document. By abstracting the essential interaction patterns common across all interfaces, AIOS 1.0 creates a consistent interaction surface that allows agents to navigate diverse computing environments. This approach significantly reduces the complexity agents face when operating in heterogeneous computer components. The agents can leverage the system-level resources, such as LLM and vector database by revoking the

²<https://modelcontextprotocol.io/introduction>

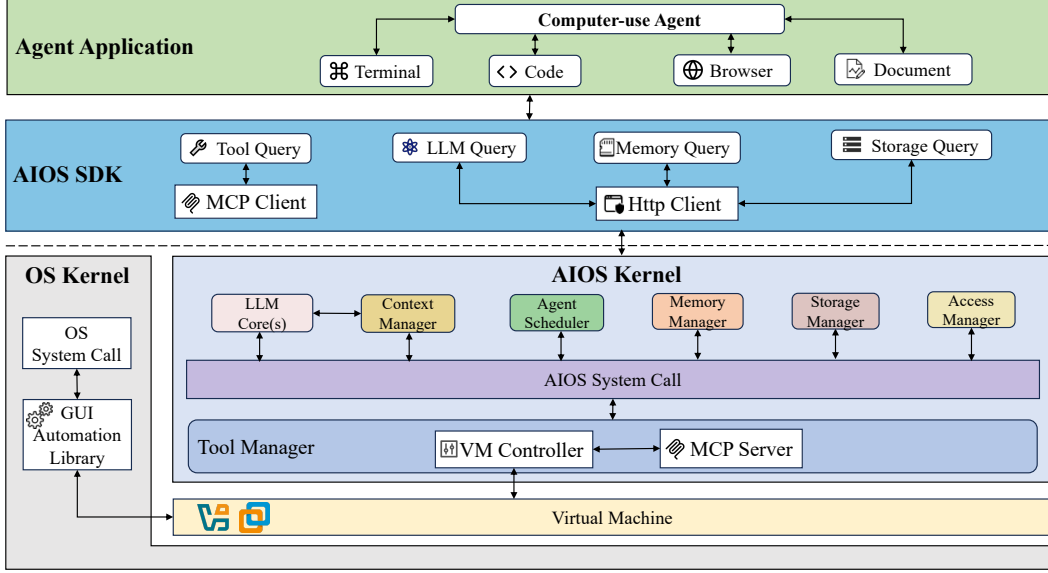


Figure 2: The serving architecture of AIOS 1.0 for computer-use agent, which extends on the basis of AIOS 0.x versions [22].

APIs within AIOS SDK [28]. While maintaining the original design from AIOS SDK regarding LLM, Memory and Storage Query APIs, AIOS 1.0 designs the integration of MCP for tool communications. MCP Client and Http Client are established as communication channels to map the agent’s semantic understanding into computer manipulation commands.

Kernel Layer: AIOS 1.0 extends the AIOS Kernel with significant enhancements focused on computer contextualization. While preserving essential components like LLM Core(s), Context Manager, and Memory Manager, we have fundamentally redesigned the Tool Manager module to incorporate a VM (Virtual Machine) Controller and MCP Server. This redesign creates a sandboxed environment that allows agents to safely interact with computer systems while maintaining a consistent semantic mapping between agent intentions and computer operations. The AIOS system calls are also complimented with Tool related system calls.

2.2 Contextualizing Computers as MCP Servers

The core innovation of AIOS 1.0 lies in its approach to transforming computers from operational tools into contextual environments that language models can naturally comprehend. As shown in Figure 3, our MCP server serves a semantic bridge between agent cognition and computer operation through a structured contextualization process:

Environment Perception Framework: Our environment perception system implements a multi-modal sensing approach that captures the computer state through complementary channels: Screenshot to provide visual signals and Accessibility Tree to provide structural information within the the current screen. Aside from the information from the visible elements in the screenshot, we also provide the mechanisms to inspect invisible information like the version information of softwares. Through these components, we provide a comprehensive semantic representation of the computing environment. The perception components allows agents to understand the computer environment as a meaningful semantic landscape rather than a collection of pixels or interface elements, benefiting their ability to reason about computer states and potential actions.

Action Space Semantics: Within AIOS 1.0, the action space is constrained into atomic but essential computer-related operations including CLICK, SCROLL, TYPE, DRAG, etc. Each action type is further encoded into the combination of several GUI control signals by leveraging the pyautogui³ library. This action space enables agents to plan and execute complex interaction sequences while reasoning about operations at a conceptual level similar to human thought processes.

³<https://pyautogui.readthedocs.io/en/latest/>

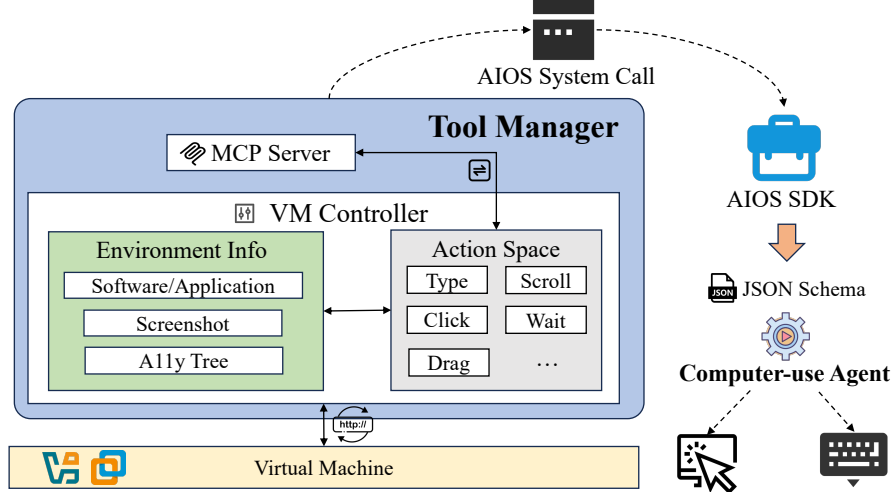


Figure 3: Pipeline for agents to take actions to interact with VM environments.

VM Controller: Considering that some of the computer-use operations can lead to irreversible outcomes, we design the VM controller to allow agents to reason and act in a safe sandbox. The HTTP interface to the Virtual Machine establishes a standardized communication protocol and serves as the abstraction of low-level operations to allow agents to interact with computer environments regardless of the underlying implementation details.

Through above contextualization, AIOS 1.0 transforms computers from operational challenges into semantic environments that language models can naturally navigate, substantially reducing the cognitive gap between language understanding and computer interaction.

3 LiteCUA: a Computer-Use Agent built on AIOS 1.0

To demonstrate the effectiveness of the AIOS 1.0 platform, we developed LiteCUA, a lightweight computer-use agent that leverages our contextualized environment architecture. We will introduce its design in the following sections.

3.1 Orchestrator-Worker Architecture

As illustrated in Figure 4, LiteCUA adopts an orchestrator-worker architecture that distributes cognitive responsibilities across specialized modules while maintaining cohesive task execution, which is inspired by some popular agent frameworks [36, 2, 18].

The Orchestrator serves as the central planning and coordination module, responsible for decomposing complex tasks into manageable subtasks, tracking progress toward goal completion. Worker modules perform specialized functions within the agent’s cognitive pipeline. This specialization allows each module to optimize for its specific function without conflating distinct cognitive processes. The modular architecture also facilitates incremental improvements to specific capabilities without requiring comprehensive system redesign.

3.2 Perceive-Reason-then-Act

In LiteCUA, we deliberately design a perceive-reason-act cognitive cycle that systematically processes computer environments through three specialized stages:

The **Perceptor** module integrates multi-modal inputs, i.e., screenshots and Ally tree to construct a comprehensive representation of the computing environment. Rather than processing raw inputs directly, the Perceptor organizes perceived elements into semantically meaningful structures that capture both the functional and relational aspects of interface elements. For example, a button is represented not merely as a visual element but as an interactive component with specific usage semantics, e.g., "Type: Button, Usage: Open the chrome".

The **Reasoner** module serves as the cognitive core of LiteCUA, interpreting the structured environment information in relation to the current task objectives. It processes textual environment

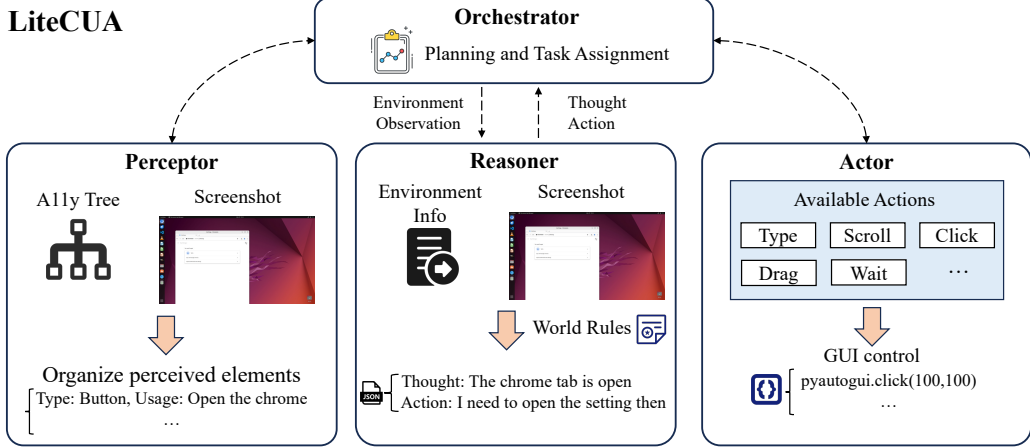


Figure 4: Design of LiteCUA. We adopt a simple orchestrator-worker architecture, where orchestrator is responsible for planning and assigning tasks and other workers deal with specialized duties like perceiving, reasoning and acting.

description alongside visual screenshot data, applying operational rules that encode interaction patterns and constraints within computing environments. These information will be fed into the Reasoner to generate both thought (interpretative understanding of the current state) and action (strategic planning for next steps) outputs.

The **Actor** module translates high-level action intentions into precise computer operations through a structured action space. It transforms semantic actions (Type, Scroll, Click, Drag, Wait) to specific GUI control commands leveraging pyautogui, e.g., `pyautogui.click(100,100)`.

This perceive-reason-act cycle enables LiteCUA to maintain goal-directed behavior across extended interaction sequences while adapting to dynamic computing environments. By leveraging AIOS 1.0’s contextualized environment architecture, we demonstrate that effective computer-use agents can be constructed with relatively simple and straightforward architectures and the performance of LiteCUA will be shown in Section 4.

4 Evaluation

4.1 Setup

Deployment Environment. The experiments are conducted on an isolated VM with 2 AMD Ryzen CPU cores and 4 GB of memory with ubuntu 22.04 desktop.

Benchmark. We evaluate the LiteCUA computer-use agent built atop of AIOS 1.0 on OSWorld [39], a benchmark which provides a diverse online environment for evaluating multi-modal agents on computer-using tasks. It consists of 369 tasks involving real-world web and desktop applications.

Baselines. For baselines, we choose three LLM baselines, i.e., GPT (4o-mini, GPT-4o) [1], and Gemini-1.5-flash [31], and three agent baselines which adopt GPT-4o as their backbones, i.e., Friday [37], Open-Interpreter [21], and AgentStore [13]. For a fair comparison, we evaluate all the methods under the "Screenshot + A11y tree" setting and only compare agents which do not introduce extra models for grounding. The metric we use is success rate provided within OSWorld.

4.2 Comparative Performance

From Table 1, LiteCUA achieves a 14.66% success rate on the OSWorld benchmark, showing modest improvements over both standalone model baselines and other agent frameworks. Compared to GPT-4o (11.21%), our approach demonstrates that properly contextualizing computer environments for LLMs can yield better results than using the model in isolation. Among agent frameworks using GPT-4o as their backbone, LiteCUA outperforms Friday (11.11%) and Open-Interpreter (8.94%), with a slight edge over AgentStore (13.55%). These results, while still showing significant room for improvement, suggest that our approach of treating computers as contextual environments provides advantages for computer-use tasks. The AIOS Agent Operating System architecture allows for more

Table 1: Success Rate Comparison

Method (Screenshot + Alloy tree)	OSWorld
LLMs	
GPT-4o-mini [1]	6.21
GPT-4o [1]	11.21
Gemini-1.5-pro [31]	5.1
Agents (based on GPT-4o)	
Friday [37]	11.11
Open-Interpreter [21]	8.94
AgentStore [13]	13.55
LiteCUA	14.66

Table 2: Detailed Performance of LiteCUA

Domain	Score	Average Steps
Chrome	7/46	28.4
GIMP	4/26	32.7
Libreoffice Calc	0/47	50.0
Libreoffice Writer	2/23	42.1
Libreoffice Impress	7/47	31.5
OS	13/24	18.2
VLC	3.11/17	35.8
Thunderbird	0/15	50.0
VSCode	8/23	25.6
Multi-app	11/101	38.4
Total	54.10/369	35.3

effective translation between LLM understanding and computer operations compared to traditional prompt-based approaches. However, the overall modest success rates across all systems (below 15%) indicate that computer-use tasks remain challenging for current AI systems, suggesting that significant architectural and modeling improvements are still needed to achieve robust performance.

4.3 Performance Breakdown by Domains

The domain-specific results reveal clear patterns in LiteCUA’s capabilities and limitations. The system performs best on OS-related tasks (13/24, 54.2%) with the lowest average step count (18.2), likely because these operations involve clearer, more structured interactions. Similarly, VSCode tasks show reasonable performance (8/23, 34.8%) with relatively efficient execution (25.6 steps on average). In contrast, the system struggles significantly with complex productivity applications, achieving no success on Libreoffice Calc (0/47) or Thunderbird (0/15). These domains typically involve more complex screenshots that need to be perceived and have specialized interaction patterns. The maximum step count (50.0) in these domains indicates that the system exhausts its operation budget without achieving the task goals. Multi-application tasks, which require maintaining context across different environments, show modest performance (11/101, 10.9%) with higher average step counts (38.4), reflecting the additional complexity of context switching. Web and media applications (Chrome and VLC) show intermediate performance.

5 Related Work

5.1 Computer-Use Agent

The automation of computer-based tasks through agents has significantly advanced with (M)LLMs, enabling them to operate across desktop, mobile, and web platforms. A key technical improvement involves moving beyond reliance on potentially noisy textual representations like HTML or accessibility trees towards more robust, vision-centric perception. For instance, systems like UGround [11] and Aria-UI [44] champion agents that perceive GUIs visually, akin to humans, and perform pixel-level operations. UGround leverages large-scale synthetic web data and adapts the LLaVA architecture for universal visual grounding, while Aria-UI employs a pure-vision approach with a scalable data pipeline to synthesize diverse instruction samples for grounding and incorporates action histories for context-aware reasoning. On mobile platforms, AutoDroid demonstrates a technical advancement by combining LLM commonsense with app-specific knowledge, acquired through automated dynamic analysis and exploration-based memory injection, to automate tasks without prior manual effort [35]. The evaluation of such computer-use agents is also evolving, with benchmarks like AgentBench [20], TheAgentCompany [40], and AgentGym [38] providing more complex and realistic scenarios for assessing their operational capabilities. Furthermore, the convergence of API-based and GUI-based interaction, creating hybrid agents, is a promising direction for versatile software automation [25].

5.2 AI Agent Operating System

The increasing complexity of LLM-based agents calls for dedicated infrastructure, leading to the development of AI Agent Operating Systems designed to provide foundational services. AIOS [22], as the first LLM agent operating system, introduces an LLM-specific kernel to manage resources and

LLM-specific services, offering agent scheduling, context management, memory and storage management, tool management, and access control, along with the agent SDK [28] for agent development. Such systems address critical challenges like efficient resource allocation, a problem also explored by research into self-resource allocation in multi-agent LLM systems where LLMs themselves can act as planners for task distribution [3]. UFO2 [49] designs and implements the desktop agent OS within the Windows environment. These developments show that agent-specific environments are crucial for creating scalable, reliable, and secure ecosystems for advanced agent applications.

6 Conclusion and Envision

In this paper, we introduced AIOS 1.0, a platform that reconceptualizes the computer-use agent challenge by focusing on environmental contextualization rather than model capabilities alone by contextualizing the computer as the MCP server. We also introduce LiteCUA, our demonstrative agent built on the AIOS 1.0 platform, which achieved competitive performance on the OSWorld benchmark with a 14.66% success rate, outperforming several specialized agent frameworks despite its relatively simple architecture.

While our current implementation shows promising results, several directions for future work remain. First, enhancing the perception framework to better capture temporal and causal relationships within computer environments could improve agent performance on complex multi-step tasks. Second, developing more sophisticated action space semantics that incorporate probabilistic reasoning could enable more robust operation in uncertain environments. Finally, extending our approach to additional computing domains, particularly those with specialized interfaces like professional creative applications, could further validate the generalizability of our contextualization methodology. AIOS 1.0 represents a step toward artificial general intelligence systems that can interact with the digital world, not by painstakingly teaching AI to use interfaces designed for humans, but by transforming those interfaces into environments that AI can naturally comprehend. This paradigm shift, from adapting agents to computers toward adapting computers for agents, offers a promising path forward for developing truly general computer-use capabilities in AI systems.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. [arXiv preprint arXiv:2303.08774](#), 2023.
- [2] Saaket Agashe, Kyle Wong, Vincent Tu, Jiachen Yang, Ang Li, and Xin Eric Wang. Agent s2: A compositional generalist-specialist framework for computer use agents. [arXiv preprint arXiv:2504.00906](#), 2025.
- [3] Alfonso Amayuelas, Jingbo Yang, Saaket Agashe, Ashwin Nagarajan, Antonis Antoniadis, Xin Eric Wang, and William Wang. Self-resource allocation in multi-agent LLM systems. [arXiv preprint arXiv:2504.02051](#), Apr 2025.
- [4] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Michal Podstawski, Hubert Niewiadomski, Piotr Nyczyk, et al. Graph of thoughts: Solving elaborate problems with large language models. [arXiv preprint arXiv:2308.09687](#), 2023.
- [5] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Buckner, et al. Windows agent arena: Evaluating multi-modal os agents at scale. [arXiv preprint arXiv:2409.08264](#), 2024.
- [6] Wenhui Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. [arXiv preprint arXiv:2211.12588](#), 2022.
- [7] Shihan Deng, Weikai Xu, Hongda Sun, Wei Liu, Tao Tan, Jianfeng Liu, Ang Li, Jian Luan, Bin Wang, Rui Yan, et al. Mobile-bench: An evaluation benchmark for llm-based mobile agents. [arXiv preprint arXiv:2407.00993](#), 2024.

- [8] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. Advances in Neural Information Processing Systems, 36, 2023.
- [9] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. arXiv preprint arXiv:2407.21783, 2024.
- [10] Yingqiang Ge, Wenyue Hua, Kai Mei, Juntao Tan, Shuyuan Xu, Zelong Li, and Yongfeng Zhang. OpenAGI: When LLM Meets Domain Experts. Advances in Neural Information Processing Systems, 36, 2023.
- [11] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for GUI agents. In International Conference on Learning Representations (ICLR), 2025. arXiv preprint arXiv:2410.05243 (2024).
- [12] Dong Guo, Faming Wu, Feida Zhu, Fuxing Leng, Guang Shi, Haobin Chen, Haoqi Fan, Jian Wang, Jianyu Jiang, Jiawei Wang, et al. Seed1. 5-vl technical report. arXiv preprint arXiv:2505.07062, 2025.
- [13] Chengyou Jia, Minnan Luo, Zhuohang Dang, Qiushi Sun, Fangzhi Xu, Junlin Hu, Tianbao Xie, and Zhiyong Wu. Agentstore: Scalable integration of heterogeneous agents as specialized generalist computer assistant. arXiv preprint arXiv:2410.18603, 2024.
- [14] Can Jin, Tong Che, Hongwu Peng, Yiyuan Li, Dimitris N. Metaxas, and Marco Pavone. Learning from teaching regularization: Generalizable correlations should be easy to imitate. In The Thirty-eighth Annual Conference on Neural Information Processing Systems, 2024.
- [15] Can Jin, Hongwu Peng, Qixin Zhang, Yujin Tang, Dimitris N Metaxas, and Tong Che. Two heads are better than one: Test-time scaling of multi-agent collaborative reasoning. arXiv preprint arXiv:2504.09772, 2025.
- [16] Mingyu Jin, Qinkai Yu, Dong Shu, Haiyan Zhao, Wenyue Hua, Yanda Meng, Yongfeng Zhang, and Mengnan Du. The impact of reasoning step length on large language models. arXiv preprint arXiv:2401.04925, 2024.
- [17] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. arXiv preprint arXiv:2401.13649, 2024.
- [18] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for "mind" exploration of large language model society. Advances in Neural Information Processing Systems, 36, 2023.
- [19] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437, 2024.
- [20] Xiao Liu, Hao Yu, Hanchen Zhang, Yaran Xu, Xuanyu Du, Xian Li, Hongyu Wang, Yaqing Chang, Yifei Zhao, Yao Liu, et al. AgentBench: Evaluating LLMs as Agents. arXiv preprint arXiv:2308.03688, Aug 2023.
- [21] Killian Lucas. Open interpreter. <https://github.com/OpenInterpreter/open-interpreter>, 2024.
- [22] Kai Mei, Xi Zhu, Wujiang Xu, Wenyue Hua, Mingyu Jin, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. AIOS: LLM Agent Operating System. arXiv preprint arXiv:2403.16971, Mar 2024.
- [23] Yichen Pan, Dehan Kong, Sida Zhou, Cheng Cui, Yifei Leng, Bing Jiang, Hangyu Liu, Yanyi Shang, Shuyan Zhou, Tongshuang Wu, et al. Webcanvas: Benchmarking web agents in online environments. arXiv preprint arXiv:2406.12373, 2024.

- [24] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. Advances in Neural Information Processing Systems, 37:126544–126565, 2024.
- [25] Yihong Qiao, Zelin Li, Yifei Wang, Shiyao Wang, Ming Scala, Wen-Tau Yih, and Lichan Hong. API-Based or GUI-Based? A Comparative Study of LLM Agents for Software Automation. arXiv preprint arXiv:2503.11069, Mar 2025.
- [26] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. ICLR, 2024.
- [27] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents. arXiv preprint arXiv:2501.12326, 2025.
- [28] Balaji Rama, Kai Mei, and Yongfeng Zhang. Cerebrum (AIOS SDK): A Platform for Agent Development, Deployment, Distribution, and Discovery. In Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (System Demonstrations), pages 135–142, 2025.
- [29] Christopher Rawles, Sarah Clinckemaulle, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, et al. Androidworld: A dynamic benchmarking environment for autonomous agents. arXiv preprint arXiv:2405.14573, 2024.
- [30] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. Advances in Neural Information Processing Systems, 36:38154–38180, 2023.
- [31] Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. Gemini: a family of highly capable multimodal models. arXiv preprint arXiv:2312.11805, 2023.
- [32] Xiaoqiang Wang and Bang Liu. Oscar: Operating system control via state-aware reasoning and re-planning. arXiv preprint arXiv:2410.18963, 2024.
- [33] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better llm agents. In Forty-first International Conference on Machine Learning, 2024.
- [34] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. Advances in Neural Information Processing Systems, 35:24824–24837, 2022.
- [35] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. AutoDroid: LLM-powered task automation in android. In Proceedings of the Annual International Conference on Mobile Computing and Networking (MobiCom), 2024. arXiv preprint arXiv:2308.15272 (2023).
- [36] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. arXiv preprint arXiv:2308.08155, 2023.
- [37] Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and Lingpeng Kong. Os-copilot: Towards generalist computer agents with self-improvement. arXiv preprint arXiv:2402.07456, 2024.
- [38] Zhisheng Xi, Yaofei Chen, Yuan Liu, Tianyu Zhang, Fei Wang, Weidong Chen, and Kai Huang. AgentGym: Evolving Large Language Model-based Agents across Diverse Environments. arXiv preprint arXiv:2312.08182, Dec 2023.

- [39] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. Advances in Neural Information Processing Systems, 37:52040–52094, 2024.
- [40] Michihiro Xu, Jewell Zhao, Justin GerLach, and Xin Wang. TheAgentCompany: A Gamified Benchmark to Evaluate AI Agents in a Simulated Software Company. arXiv preprint arXiv:2402.00811, Feb 2024.
- [41] Qiantong Xu, Fenglu Hong, Bo Li, Changran Hu, Zhengyu Chen, and Jian Zhang. On the tool manipulation capability of open-source large language models. arXiv preprint arXiv:2305.16504, 2023.
- [42] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2. 5 technical report. arXiv preprint arXiv:2412.15115, 2024.
- [43] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering.
- [44] Yuhao Yang, Yue Wang, Dongxu Li, Ziyang Luo, Bei Chen, Chao Huang, and Junnan Li. Aria-UI: Visual Grounding for GUI Instructions. arXiv preprint arXiv:2412.16256, Dec 2024.
- [45] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. Advances in Neural Information Processing Systems, 35:20744–20757, 2022.
- [46] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. arXiv preprint arXiv:2305.10601, 2023.
- [47] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. International Conference on Learning Representations, 2023.
- [48] Xiao Yu, Baolin Peng, Vineeth Vajipey, Hao Cheng, Michel Galley, Jianfeng Gao, and Zhou Yu. Exact: Teaching ai agents to explore with reflective-mcts and exploratory learning. arXiv preprint arXiv:2410.02052, 2024.
- [49] Chaoyun Zhang, He Huang, Chiming Ni, Jian Mu, Si Qin, Shilin He, Lu Wang, Fangkai Yang, Pu Zhao, Chao Du, et al. Ufo2: The desktop agents. arXiv preprint arXiv:2504.14603, 2025.
- [50] Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges. arXiv preprint arXiv:2401.07339, 2024.
- [51] Zhisong Zhang, Tianqing Fang, Kaixin Ma, Wenhao Yu, Hongming Zhang, Haitao Mi, and Dong Yu. Enhancing web agents with explicit rollback mechanisms. arXiv preprint arXiv:2504.11788, 2025.
- [52] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. arXiv preprint arXiv:2307.13854, 2023.